

Maestría en Computación de Borde

Desarrollo de Aplicaciones para Internet de las Cosas

TRABAJO PRÁCTICO INTEGRADOR

Nodo de monitoreo ambiental para un depósito de frutas y verduras

Un ESP32-C3 mide temperatura y humedad, las publica por MQTT sobre TLS mutuo, y el panel web las muestra y le devuelve el umbral de alerta al nodo.

ALUMNO	Esp. Ing. Joaquín Manuel Beitia
DOCENTES	Leopoldo Zimperz · Mariano Bustos
CURSADA	Bimestre 3B2026 — modalidad asincrónica
LUGAR Y FECHA	Ciudad de Rosario, septiembre de 2026

Un sistema que ya funciona, sin nada que mida el mundo físico

Tienda de Frutas Luján: gestión de pedidos de una distribuidora, en producción sobre un VPS.

Lo que ya está

API Node/Express + TypeScript + Prisma sobre PostgreSQL, panel Angular, toma de pedidos por WhatsApp e impresión de tickets.

Lo que falta

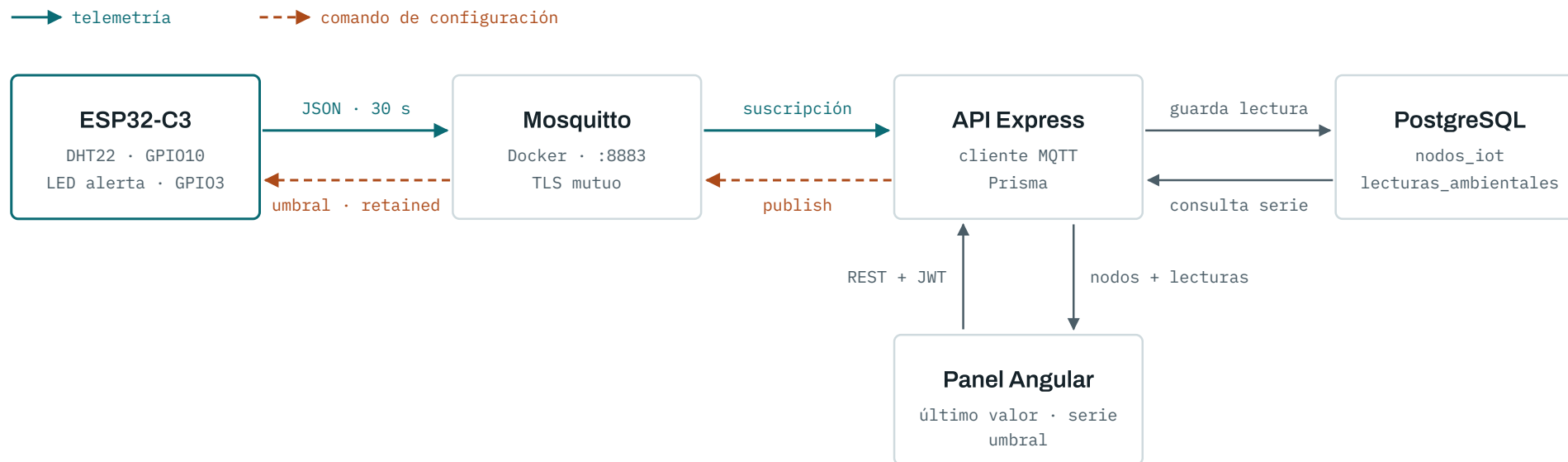
La capa de percepción y transporte. No hay sensores, no hay MQTT, no hay nodos. Todo el dato entra tipeado por una persona.

Medir el depósito donde se acopia la mercadería antes del reparto

Para fruta y verdura, la temperatura del depósito o de la cámara frigorífica es la variable que decide si la mercadería llega bien.

- **Componente nuevo**, desarrollado íntegramente en local e independiente del sistema en producción.
- **Mismo stack** que el sistema real (Express + TypeScript, Prisma, Angular), para que quede disponible ante una integración futura.
- **Bidireccional**: el nodo no solo reporta, también recibe configuración desde el panel.

ARQUITECTURA



Todo el tramo MQTT viaja cifrado y con certificado de cliente. El único componente físico es el ESP32-C3; broker, API, base y panel corren en Docker sobre la misma máquina.

Tres tópicos, jerarquía `lujan/deposito/{nodo}/...`

TÓPICO	DIRECCIÓN	PAYLOAD
<code>.../telemetria</code>	nodo → backend	<code>{"temperatura": 4.2, "humedad": 68.5, "ts": "2026-08-26T18:00:00Z"}</code>
<code>.../config</code>	backend → nodo	<code>{"umbral_alerta": 8.0}</code> · retained
<code>.../estado</code>	nodo → backend	online / offline · last will

El backend se suscribe con comodín a `lujan/deposito/+/telemetria` : sumar un segundo nodo no requiere tocar el backend, alcanza con emitir otro código de nodo.

El camino de una lectura

- 01 **El DHT22 mide.** Si la lectura vuelve inválida, el nodo la descarta y no publica.
- 02 **El ESP32-C3 arma el JSON** con la hora sincronizada por NTP y lo publica por MQTT sobre TLS.
- 03 **Mosquitto valida el certificado del nodo** y entrega el mensaje a los suscriptos.
- 04 **El backend valida el payload:** tipos correctos y timestamp parseable, o descarta y loguea.
- 05 **Prisma escribe:** upsert del nodo con su `ultimo_visto` e insert de la lectura.
- 06 **El panel refresca** los nodos cada 15 s y la serie temporal cada 30 s.

Cambiar el umbral desde el panel prende un LED en el depósito

-
- 01 El panel llama `PUT /api/iot/nodos/nodo01/umbral` con JWT; `readonly` no puede.
-
- 02 La API persiste el umbral en el nodo y guarda el cambio en el historial, en una transacción.
-
- 03 Publica en `.../config` con `retain`, así el nodo recibe el último umbral apenas se suscribe, aunque se haya reiniciado.
-
- 04 El nodo actualiza el umbral en caliente, sin flashear ni reiniciar.
-
- 05 En la próxima lectura, si la temperatura llegó al umbral, prende el LED de alerta.
-

El broker no acepta a nadie sin certificado

TLS mutuo, y nada más

Un único listener en el 8883. Sin puerto en texto plano y sin acceso anónimo:

```
require_certificate true .
```

CA propia

Un script genera la CA y firma tres pares: servidor, backend y `nodo01`. Claves y certificados no se versionan; sí el script que los genera.

Dos peleas con mbedTLS

El ESP32 no habla TLS 1.3, así que hubo que forzar 1.2 en el broker; y valida la IP contra SAN de tipo `DNS`, no `IP`. Con clientes de escritorio ninguna de las dos aparece.

Un nodo alimentado por USB, colgado del WiFi

Placa

ESP32-C3. La propuesta decía S3; al verificar con `esptool` resultó ser una C3, funcionalmente equivalente para este caso.

Sensor

DHT22 pelado, de cuatro patas, con pull-up de 10 k Ω entre VCC y DATA. DATA en GPIO10.

Indicador

LED con resistencia de 300 Ω en GPIO3. Se evitaron GPIO 2, 8 y 9 porque definen el modo de arranque, y 20/21 porque son el puerto serie.

Funcionando de punta a punta con hardware real

Cuatro contenedores y una placa. El entorno completo se levanta con un comando.

Levantar todo

```
./mosquitto/certs/generate-certs.sh  
docker compose up -d
```

Postgres, Mosquitto, la API y el panel. Las migraciones de Prisma se aplican solas al arrancar la API.

Verificado

El nodo publica cada 30 s desde el WiFi de casa, la lectura llega a Postgres, el panel la grafica, y el LED responde al cambio de umbral sin tocar el firmware.